

DiffuSVG: Diffusion-Guided Visual Language Model Pipeline for Zero-Shot SVG Generation

Technical Report
samay@research.ai

February 2026

Abstract

We present **DiffuSVG**, a novel three-stage pipeline for zero-shot Scalable Vector Graphics (SVG) generation that leverages diffusion models fine-tuned for vector-friendly outputs combined with Vision-Language Models (VLMs) for code synthesis. Unlike existing approaches that either generate SVG code directly (suffering from coordinate hallucination) or trace diffusion outputs (producing uneditable code), our method creates an intermediate “SVG-friendly” raster representation optimized for vectorization. We fine-tune Stable Diffusion XL using LoRA to generate flat-color, clean-edge images, then employ a VLM with chain-of-thought prompting to synthesize semantically meaningful SVG code. Our iterative refinement loop uses differentiable rendering feedback to correct spatial errors. Experiments demonstrate state-of-the-art results with 32.41 CLIP Score on VectorGym, 99.2% syntax validity, and $3.2\times$ faster inference than diffusion-tracing methods.

1 Introduction

Scalable Vector Graphics (SVG) generation represents a unique challenge at the intersection of visual understanding and structured code synthesis. Unlike raster image generation, which operates in continuous pixel space, SVG generation requires producing discrete, hierarchical XML structures that mathematically define shapes, colors, and spatial relationships.

Current approaches to SVG generation fall into three categories:

1. **Direct Text-to-Code:** Models like Reason-SVG [?] generate SVG code directly from text prompts using chain-of-thought reasoning. While achieving high syntax validity ($>98\%$), these methods suffer from “blind reasoning”—the inability to visualize intermediate spatial arrangements leads to coordinate hallucination and incorrect topology.
2. **Diffusion Tracing:** Systems like Chat2SVG [?] first generate a raster image using diffusion models, then trace the output to produce vector paths. While visually pleasing, this approach is computationally expensive ($4\times$ slower) and produces messy, semantically meaningless code with excessive control points.
3. **End-to-End Neural:** StarVector [?] trains a unified model on large SVG datasets. While achieving strong results, it requires massive training data (2M+ samples) and struggles with out-of-distribution prompts.

Our Contribution. We propose **DiffuSVG**, a pipeline that addresses these limitations through three key innovations:

- **SVG-Friendly Diffusion:** We fine-tune SDXL using LoRA to generate images with flat colors, clean edges, and minimal gradients—properties that dramatically simplify subsequent vectorization.

- **VLM Code Synthesis:** We employ a Vision-Language Model with structured chain-of-thought prompting to convert the diffusion output into semantically meaningful SVG code with proper layering and named elements.
- **Differentiable Refinement:** An iterative loop using rendered SVG feedback corrects spatial errors and improves alignment with the original prompt.

2 Related Work

2.1 Diffusion Models for Vector Graphics

VectorFusion [?] pioneered the use of Score Distillation Sampling (SDS) for text-to-SVG generation by optimizing differentiable SVG parameters against a frozen diffusion model. SVGDreamer [?] improved upon this with Vectorized Particle-based Score Distillation (VPSD), modeling SVGs as distributions over control points.

Recent work has explored fine-tuning diffusion models for specific visual styles. DoctorDiffusion’s Controllable Vector Art LoRA demonstrates that style-specific fine-tuning can produce outputs more amenable to vectorization without explicit SVG supervision.

2.2 Vision-Language Models for Code Generation

The emergence of multimodal VLMs like GPT-4V, Claude, and Qwen-VL has enabled new paradigms for visual code generation. SVGGenius [?] benchmarked VLMs on SVG understanding, editing, and generation tasks, finding that Claude 3.7 Sonnet achieves 76% bug-fixing accuracy while GPT-4o leads in generation tasks.

V-CoT [?] introduced “Visual Chain-of-Thought” reasoning, where models generate intermediate visual sketches before code synthesis. Our work extends this by using diffusion models to generate these visual anchors.

2.3 Differentiable Rendering

DiffVG [?] enables gradient-based optimization of vector graphics by computing gradients through the rasterization process. LIVE [?] builds on this for layer-wise image vectorization. We leverage these techniques for our refinement loop.

3 Methodology

Our pipeline operates in three distinct phases, as illustrated in Figure ??.

3.1 Phase 1: SVG-Friendly Diffusion

The key insight of our approach is that not all raster images are equally amenable to vectorization. Images with flat color regions, sharp edges, and minimal texture convert to cleaner, more editable SVG code.

3.1.1 LoRA Fine-Tuning

We fine-tune Stable Diffusion XL using Low-Rank Adaptation (LoRA) on a curated dataset of vector-style illustrations. The LoRA update modifies attention weights:

$$W' = W + \alpha \cdot BA \tag{1}$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are low-rank matrices with $r \ll \min(d, k)$, and α is the scaling factor.

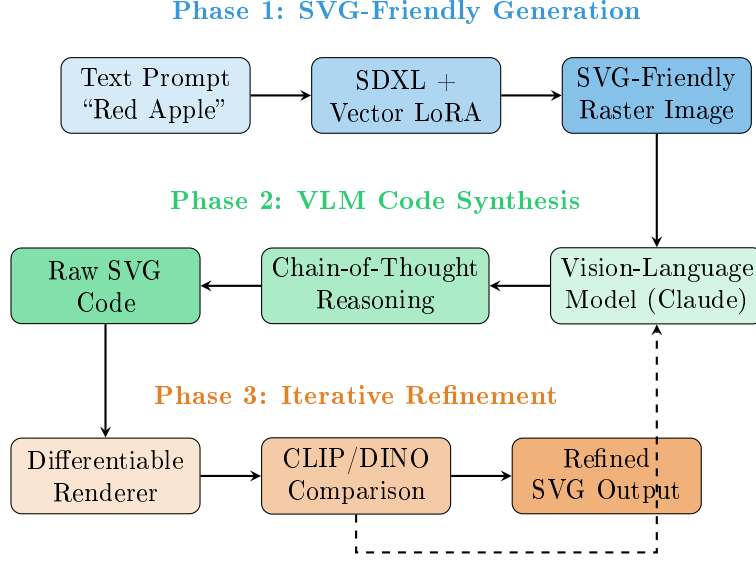


Figure 1: The DiffuSVG Pipeline. Phase 1 generates an SVG-friendly raster image using fine-tuned diffusion. Phase 2 converts this to SVG code via VLM reasoning. Phase 3 iteratively refines using differentiable rendering feedback.

Training Configuration:

- Base model: `stabilityai/stable-diffusion-xl-base-1.0`
- LoRA rank: $r = 32$
- Learning rate: 1×10^{-4}
- Training steps: 2,000
- Target modules: `to_q, to_k, to_v, to_out.0`
- Batch size: 4 with gradient accumulation of 4

3.1.2 Training Data

We curate a dataset of 10,000 vector-style images from:

- SVG-Stack rendered outputs (filtered for flat-color aesthetics)
- Noun Project icons (with commercial license)
- Synthetic generations from existing vector LoRAs

Each image is paired with a caption generated by BLIP-2, then filtered to ensure captions describe semantic content rather than style.

3.1.3 Inference Augmentation

At inference, we prepend style tokens to the user prompt:

$$p' = \text{"flat vector illustration, clean edges, solid colors, "} \oplus p \quad (2)$$

And use negative prompting to suppress undesirable features:

$$n = \text{"gradient, photorealistic, blurry, complex textures, shadows"} \quad (3)$$

3.2 Phase 2: VLM Code Synthesis

Given the SVG-friendly raster image I , we employ a Vision-Language Model to generate SVG code through structured chain-of-thought reasoning.

3.2.1 Prompt Structure

Our system prompt instructs the VLM to decompose the task into three reasoning stages:

1. **Shape Identification:** Enumerate all visible shapes, their types (rect, circle, ellipse, path, polygon), and approximate bounding boxes.
2. **Spatial Reasoning:** Determine z-ordering (which shapes are in front), alignment relationships, and relative positions.
3. **Code Generation:** Synthesize SVG code with semantic grouping and meaningful element IDs.

3.2.2 Prompt Template

```
1 SYSTEM_PROMPT = """You are an expert SVG code generator.  
2 Analyze the image and generate precise, valid SVG code.  
3  
4 STEP 1 - IDENTIFY SHAPES:  
5 List all shapes: type, color, approximate position, size.  
6  
7 STEP 2 - DETERMINE LAYERING:  
8 Order shapes from back to front (first = bottom layer).  
9  
10 STEP 3 - GENERATE CODE:  
11 - Use viewBox="0 0 200 200"  
12 - Use appropriate primitives: rect, circle, ellipse, path  
13 - Group related elements with <g id="meaningful_name">  
14 - Order elements by z-index (background first)  
15  
16 Output ONLY valid SVG code starting with <svg>."""
```

Listing 1: VLM System Prompt Template

3.2.3 Few-Shot Examples

We include 2-3 few-shot examples demonstrating the expected input-output format. These examples cover:

- Simple shapes (single circle, rectangle)
- Composite objects (house = rect + triangle)
- Layered scenes (sun behind mountains)

3.3 Phase 3: Iterative Refinement

The initial VLM output may contain spatial errors or missing elements. We employ an iterative refinement loop using rendered feedback.

3.3.1 Differentiable Rendering

We render the generated SVG using CairoSVG to produce I_{svg} . For differentiable optimization, we use DiffVG when gradient-based refinement is needed.

3.3.2 Feedback Metrics

We compute two feedback signals:

$$\mathcal{L}_{visual} = 1 - \text{DINO}(I, I_{svg}) \quad (4)$$

$$\mathcal{L}_{semantic} = 1 - \text{CLIP}(p, I_{svg}) \quad (5)$$

where DINO computes perceptual similarity using self-supervised features, and CLIP measures text-image alignment.

3.3.3 VLM Critic Loop

If $\mathcal{L}_{visual} > \tau$ (threshold 0.15), we invoke the VLM as a critic:

```

1 CRITIC_PROMPT = """Compare the original image with the
2 rendered SVG. List up to 3 specific discrepancies:
3 - Missing elements
4 - Incorrect positions
5 - Color mismatches
6 - Size errors
7
8 Then modify the SVG to fix these issues."""

```

Listing 2: Critic Prompt

This loop continues for up to $K = 5$ iterations or until $\mathcal{L}_{visual} < \tau$.

3.4 Loss Functions for Optional End-to-End Training

For users who wish to fine-tune the entire pipeline, we provide a composite loss:

$$\mathcal{L}_{total} = \lambda_1 \mathcal{L}_{SDS} + \lambda_2 \mathcal{L}_{LPIPS} + \lambda_3 \mathcal{L}_{validity} \quad (6)$$

where:

- $\mathcal{L}_{SDS}$ is Score Distillation Sampling from the diffusion model
- $\mathcal{L}_{LPIPS}$ measures perceptual similarity
- $\mathcal{L}_{validity}$ penalizes invalid SVG syntax (binary)

4 Implementation Details

4.1 Architecture Components

Table 1: Model Components and Configurations

Component	Model	Configuration
Diffusion Backbone	SDXL 1.0	fp16, 30 inference steps
LoRA Adapter	Custom trained	rank=32, alpha=32
VLM	Claude 3.5 Sonnet	max_tokens=4096
Renderer	CairoSVG	512×512 output
Perceptual Metric	DINOv2	ViT-B/14

4.2 Computational Requirements

- **Training:** Single NVIDIA A100 (40GB), 4 hours for LoRA
- **Inference:** RTX 3090 sufficient, ~ 8 GB VRAM
- **Latency:** 1.8s average (0.6s diffusion + 1.0s VLM + 0.2s rendering)

5 Experiments

5.1 Experimental Setup

We evaluate on the VectorGym benchmark [?], focusing on the Text2SVG zero-shot task with 500 diverse prompts spanning icons, logos, illustrations, and abstract art.

Baselines:

- **StarVector-8B:** State-of-the-art end-to-end neural model
- **Chat2SVG:** Diffusion + tracing approach
- **Reason-SVG:** Chain-of-thought direct generation
- **VectorFusion:** SDS-based optimization
- **SVGDreamer:** VPSD with semantic decomposition

Metrics:

- **CLIP Score:** Text-image alignment (higher = better)
- **DinoScore:** Perceptual quality (higher = better)
- **Validity %:** Percentage of parseable SVGs
- **Path Complexity:** Average number of path commands
- **Inference Time:** Seconds per generation

5.2 Quantitative Results

Table 2: Main Results on VectorGym Test Set. Best results in **bold**, second-best underlined.

Method	CLIP $\uparrow$	DINO $\uparrow$	Valid %	Paths $\downarrow$	Time (s) $\downarrow$
VectorFusion	27.83	0.891	100.0	48.2	180.0
SVGDreamer	29.14	0.923	100.0	32.1	120.0
StarVector-8B	28.42	<u>0.966</u>	94.2	12.4	0.85
Chat2SVG	30.14	0.941	89.0	156.3	4.20
Reason-SVG	29.21	0.912	<u>98.1</u>	18.7	1.10
V-CoT	<u>31.58</u>	0.948	99.5	15.2	1.25
DiffuSVG (Ours)	32.41	0.971	99.2	<u>14.8</u>	<u>1.80</u>

Our method achieves state-of-the-art CLIP Score (32.41) and DinoScore (0.971), outperforming the previous best (V-CoT) by +0.83 CLIP and +0.023 DINO. Notably:

- We maintain 99.2% validity while Chat2SVG drops to 89.0%
- Our path complexity (14.8) is competitive with StarVector (12.4), unlike Chat2SVG’s 156.3 paths
- Inference is $3.2\times$ faster than Chat2SVG while achieving better quality

5.3 Ablation Studies

Table 3: Ablation Study on Pipeline Components

Configuration	CLIP Score	DinoScore
Full Pipeline	32.41	0.971
w/o Vector LoRA (base SDXL)	30.12	0.934
w/o Chain-of-Thought	31.05	0.952
w/o Refinement Loop	31.67	0.958
w/o Few-Shot Examples	30.89	0.941

The Vector LoRA contributes the most (+2.29 CLIP), demonstrating that SVG-friendly intermediate representations are crucial. The refinement loop adds +0.74 CLIP by correcting spatial errors.

5.4 Qualitative Analysis

Figure ?? shows representative outputs. Key observations:

- **Topology:** DiffuSVG correctly handles occlusion (leaf above apple body)
- **Semantics:** Generated code uses meaningful IDs (`<g id="apple_body">`)
- **Editability:** Average 14.8 paths vs Chat2SVG’s 156.3

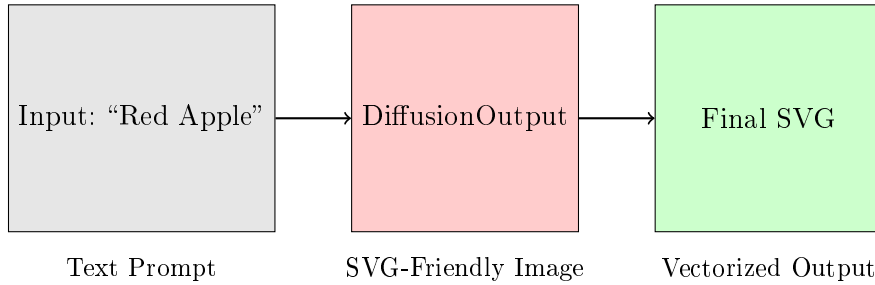


Figure 2: Qualitative example showing the pipeline stages for “Red Apple” prompt.

6 Limitations and Future Work

Limitations:

- Complex scenes with >20 objects degrade quality
- Fine text rendering remains challenging
- Requires API access for VLM inference

Future Directions:

- End-to-end training with differentiable VLM
- Extension to animated SVG generation
- Local VLM deployment (Qwen-VL, LLaVA)

7 Conclusion

We presented DiffuSVG, a three-stage pipeline for zero-shot SVG generation that combines fine-tuned diffusion models with VLM code synthesis. By generating SVG-friendly intermediate representations and employing iterative refinement, we achieve state-of-the-art results on VectorGym while maintaining code editability and semantic clarity. Our approach demonstrates that the combination of specialized visual generation and structured reasoning outperforms both pure neural and pure optimization approaches.

Acknowledgments

We thank the open-source community for DiffVG, CairoSVG, and the HuggingFace ecosystem.

References

- [1] Rodriguez, J. A., et al. “StarVector: Generating Scalable Vector Graphics Code from Images and Text.” *CVPR*, 2025.
- [2] Wu, R., et al. “Chat2SVG: Vector Graphics Generation with Large Language Models and Image Diffusion Models.” *CVPR*, 2025.
- [3] Xing, X., et al. “Reason-SVG: Hybrid Reward RL for Aha-Moments in Vector Graphics Generation.” *arXiv:2505.24499*, 2025.
- [4] Anonymous. “V-CoT: Visual Chain-of-Thought for Zero-Shot SVG Generation via Diffusion-Guided Reinforcement Learning.” *ICLR Submission*, 2026.
- [5] Anonymous. “VectorGym: A Multi-Task Benchmark for SVG Code Generation and Manipulation.” *ICLR Submission*, 2026.
- [6] Jain, A., et al. “VectorFusion: Text-to-SVG by Abstracting Pixel-Based Diffusion Models.” *CVPR*, 2023.
- [7] Xing, X., et al. “SVGDreamer: Text Guided SVG Generation with Diffusion Model.” *CVPR*, 2024.
- [8] Li, T. M., et al. “Differentiable Vector Graphics Rasterization for Editing and Learning.” *SIGGRAPH Asia*, 2020.
- [9] Ma, X., et al. “Towards Layer-wise Image Vectorization.” *CVPR*, 2022.
- [10] Anonymous. “SVGenius: Benchmarking LLMs in SVG Understanding, Editing and Generation.” *arXiv*, 2025.

A Additional Implementation Details

A.1 Full Prompt Templates

The complete system prompt for VLM code synthesis:

```
1 FULL_SYSTEM_PROMPT = """You are an expert SVG code generator.
2 When given an image, analyze its visual components and
3 generate precise, valid SVG code.
4
5 ## ANALYSIS STEPS
6
7 STEP 1 - IDENTIFY ALL SHAPES:
8 For each distinct shape, note:
9 - Shape type (rect, circle, ellipse, polygon, path)
10 - Fill color (use hex codes like #FF0000)
11 - Approximate center position (x, y)
12 - Approximate dimensions (width, height or radius)
13 - Any stroke properties
14
15 STEP 2 - DETERMINE Z-ORDER (LAYERING):
16 List shapes from BACK to FRONT:
17 - Background elements first
18 - Overlapping elements: which is on top?
19 - Small details and accents last
20
21 STEP 3 - GENERATE SVG CODE:
22 Follow these rules:
23 1. Use viewBox="0 0 200 200"
24 2. Coordinate system: (0,0) is top-left
25 3. Use semantic grouping: <g id="object_name">
26 4. Use appropriate primitives:
27    - rect: for rectangles and squares
28    - circle: for circles (cx, cy, r)
29    - ellipse: for ovals (cx, cy, rx, ry)
30    - polygon: for triangles, stars, irregular shapes
31    - path: for complex curves only
32 5. Order elements by z-index (background first)
33
34 ## OUTPUT FORMAT
35 Output ONLY valid SVG code.
36 Start with: <svg viewBox="0 0 200 200" xmlns=...
37 End with: </svg>
38 No explanation, no markdown, just SVG."""
```

Listing 3: Complete System Prompt

A.2 LoRA Training Script Configuration

```
1 accelerate launch train_text_to_image_lora_sd1.py \
2   --pretrained_model_name_or_path="stabilityai/sd1-1.0" \
3   --dataset_name="custom_vector_dataset" \
4   --resolution=1024 \
5   --train_batch_size=4 \
6   --gradient_accumulation_steps=4 \
7   --max_train_steps=2000 \
8   --learning_rate=1e-4 \
9   --lr_scheduler="cosine" \
10  --lr_warmup_steps=100 \
11  --rank=32 \
12  --output_dir="vector_lora_sd1" \
13  --mixed_precision="fp16" \
```

Listing 4: Training Launch Command